Practical 12: React Basics and Component Architecture

Aim

To study and implement basic React concepts such as components, props, and state using a Counter application.

Theory

React is a declarative, component-based JavaScript library (developed by Meta) for building user interfaces out of small, reusable, self-contained pieces called components, instead of manipulating the DOM directly and imperatively as in vanilla JavaScript. A functional component is simply a JavaScript function that accepts props (inputs) and returns JSX - a syntax extension that lets HTML-like markup be written directly inside JavaScript, which is compiled (transpiled) into regular JavaScript function calls (React.createElement) by tools like Babel. Props (short for properties) are read-only data passed from a parent component to a child component, enabling component reuse with different inputs. The useState hook is one of React's built-in Hooks, which let functional components hold local, mutable state without needing class components; calling its setter function (e.g., setCount) does not mutate the variable in place but instead schedules a re-render of the component with the updated value, and React efficiently updates only the changed parts of the real DOM using its Virtual DOM diffing algorithm. Component composition, as seen with the Counter component nested inside the App component, allows complex UIs to be built by combining small, independently testable pieces, one of React's core architectural strengths.

Algorithm

1. Import React and the useState hook.
2. Define a Counter component that initializes a count state variable to 0 using useState.
3. Render the current count and a button that calls setCount to increment the count on click.
4. Define an App component that renders a heading and includes the Counter component.
5. Export App as the default component and render it in the browser.
6. Verify that each click on the Increment button updates the displayed count.

Program

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

function App() {
  return (
    <div>
      <h1>React Basics Demo</h1>
      <Counter />
    </div>
  );
}

export default App;
```

Output

```
Browser renders:
  React Basics Demo
  Count: 0
  [ Increment ]

Each click on "Increment" re-renders the Counter
component and increases the displayed count by 1:
  Count: 1
  Count: 2
  Count: 3 ...
```

Result / Conclusion

Basic React concepts - components, props, and state management using useState - were successfully implemented and verified in the Counter application.

Viva Questions

Q1. What is the Virtual DOM and why does React use it?

Ans: The Virtual DOM is a lightweight JavaScript copy of the real DOM. When state changes, React creates a new Virtual DOM tree and diffs it against the previous one (reconciliation), identifying the minimal set of real DOM changes needed. It then applies only those changes (patching), which is much faster than re-rendering the entire page. This makes React updates efficient, even for complex UIs with many components.

Q2. What is the difference between props and state in React?

Ans: Props (properties) are read-only inputs passed from a parent component to a child — the child cannot modify them. They are used for component configuration and data flow downwards. State is mutable data managed inside a component using useState — when state changes, React re-renders that component. Props flow down; state is local. State changes trigger re-renders; prop changes cause the parent to re-render, which may update the child's props.

Q3. What is JSX and how does it work?

Ans: JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup inside JavaScript code. It is NOT valid JavaScript — a transpiler (Babel) converts JSX to React.createElement() calls. For example, <h1>Hello</h1> becomes React.createElement('h1', null, 'Hello'). JSX allows embedding JavaScript expressions using curly braces {expression}. It makes component code more readable and maintainable than raw React.createElement() calls.

Q4. What does useState return and how do you use it?

Ans: useState(initialValue) returns an array of exactly two elements: [currentValue, setterFunction]. Destructuring: const [count, setCount] = useState(0). count holds the current state value (0 initially). setCount(newValue) schedules a re-render with the new value — it does NOT mutate count directly. Each component instance has its own independent state, and state persists across re-renders (unlike regular variables which reset).

Q5. What is the difference between React and vanilla JavaScript for building UIs?

Ans: Vanilla JS manually manipulates the DOM (document.getElementById, innerHTML), which becomes complex and error-prone for large, dynamic UIs. React uses declarative programming — you describe what the UI should look like for a given state, and React handles DOM updates. React provides: component reuse, unidirectional data flow, efficient updates via Virtual DOM, and a rich ecosystem. Trade-off: React adds a framework overhead (~40KB); vanilla JS is lighter for simple pages.